

CS23331-DAA-2024-CSE / 1-Number of Zeros in a Given Array

1-Number of Zeros in a Given Array

Started on	Friday, 19 September 2025, 1:30 PM
State	Finished
Completed on	Friday, 19 September 2025, 1:46 PM
Time taken	16 mins 14 secs
Marks	1.00/1.00
Grade	10.00 out of 10.00 (100%)

Question 1 | Correct | Mark 1.00 out of 1.00 |  [Flag question](#)

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 int findFirstZero(int arr[], int low, int high) {
3     int firstZeroIndex = -1;
4     while (low <= high) {
5         int mid = low + (high - low) / 2;
6
7         if (arr[mid] == 0) {
8             firstZeroIndex = mid;
9             high = mid - 1;
10        } else {
11            low = mid + 1;
12        }
13    }
14    return firstZeroIndex;
15 }
16 int countZeroes(int arr[], int size) {
17     int firstZeroIndex = findFirstZero(arr, 0, size - 1);
18     if (firstZeroIndex == -1) {
19         return 0;
20     }
21     return size - firstZeroIndex;
22 }
23
24 int main() {
25     int m;
26     scanf("%d", &m);
27     int arr[m];
28     for (int i = 0; i < m; i++) {
29         scanf("%d", &arr[i]);
30     }
31     int zeroCount = countZeroes(arr, m);
32     printf("%d\n", zeroCount);
33     return 0;
34 }
35
```

	Input	Expected	Got	
✓	5 1 1 1 0 0	2	2	✓
✓	10 1 1 1 1 1 1 1 1 1 1	0	0	✓

2-Majority Element

Started on	Friday, 19 September 2025, 1:49 PM
State	Finished
Completed on	Friday, 19 September 2025, 2:29 PM
Time taken	39 mins 8 secs
Marks	1.00/1.00
Grade	10.00 out of 10.00 (100%)

Question 1 | [Correct](#) Mark 1.00 out of 1.00 [Flag question](#)

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than $\lfloor n / 2 \rfloor$ times. You may assume that the majority element always exists in the array.

Example 1:

Input: `nums = [3,2,3]`

Output: 3

Example 2:

Input: `nums = [2,2,1,1,1,2,2]`

Output: 2

Constraints:

- `n == nums.length`
- $1 \leq n \leq 5 \cdot 10^4$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main() {
4     int n;
5     scanf("%d", &n);
6     int nums[n];
7     for (int i = 0; i < n; i++) {
8         scanf("%d", &nums[i]);
9     }
10
11     int candidate = 0;
12     int count = 0;
13
14     for (int i = 0; i < n; i++) {
15         if (count == 0) {
16             candidate = nums[i];
17             count = 1;
18         } else if (nums[i] == candidate) {
19             count++;
20         } else {
21             count--;
22         }
23     }
24
25     printf("%d\n", candidate);
26
27     return 0;
28 }
```

	Input	Expected	Got	
✓	3 3 2 3	3	3	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Finish review](#)

[Back to Course](#)

3-Finding Floor Value

Started on	Sunday, 12 October 2025, 9:13 AM
State	Finished
Completed on	Sunday, 12 October 2025, 9:22 AM
Time taken	9 mins 23 secs
Marks	1.00/1.00
Grade	10.00 out of 10.00 (100%)

Question 1 | Correct Mark 1.00 out of 1.00  [Flag question](#)

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer: (penalty regime: 0 %)

```

1 #include<stdio.h>
2 int findFloor(int arr[],int n,int x){
3     int low=0;
4     int high=n-1;
5     int floor_val=-1;
6
7     if(x<arr[0])
8         return -1;
9     while(low<=high){
10         int mid=low+(high-low)/2;
11         if(arr[mid]==x)
12             return arr[mid];
13         if(arr[mid]<x){
14             floor_val=arr[mid];
15             low=mid+1;
16         }
17         else{
18             high=mid-1;
19         }
20     }
21     return floor_val;
22 }
23 int main(){
24     int n;
25     scanf("%d",&n);
26     int arr[n];
27     for(int i=0;i<n;i++){
28         scanf("%d",&arr[i]);
29     }
30     int x;
31     scanf("%d",&x);
32     int result=findFloor(arr,n,x);
33     if(result== -1)
34         printf("Floor does not exist\n");
35     else
36         printf("%d\n",result);
37     return 0;
38 }
39

```

	Input	Expected	Got	
✓	6 1 2 8 10 12 19 5	2	2	✓
✓	5 10 22 85 108	85	85	✓

	129			
	100			
✓	7	9	9	✓
	3			
	5			
	7			
	9			
	11			
	13			
	15			
	10			

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Finish review](#)

[Back to Course](#)

4-Two Elements sum to x

Started on	Sunday, 12 October 2025, 9:23 AM
State	Finished
Completed on	Sunday, 12 October 2025, 9:32 AM
Time taken	9 mins 13 secs
Marks	1.00/1.00
Grade	10.00 out of 10.00 (100%)

Question 1 | [Correct](#) [Mark 1.00 out of 1.00](#) [Flag question](#)

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int findPair(int arr[],int left,int right,int x){
3     if(left>=right)
4     | return 0;
5     int current_sum=arr[left]+arr[right];
6     if(current_sum==x){
7         printf("%d\n",arr[left]);
8         printf("%d\n",arr[right]);
9         return 1;
10    }
11    else if(current_sum<x){
12        | return findPair(arr,left+1,right,x);
13    }
14    else{
15        | return findPair(arr,left,right-1,x);
16    }
17 }
18 int main(){
19     int n;
20     scanf("%d",&n);
21     int arr[n];
22     for(int i=0;i<n;i++)
23     | scanf("%d",&arr[i]);
24     int x;
25     scanf("%d",&x);
26     if(!findPair(arr,0,n-1,x))
27     | printf("No\n");
28     return 0;
29 }
30 }
31
32
```

	Input	Expected	Got	
✓	4 2 4 8 10 14	4 10	4 10	✓
✓	5 2 4 6 8 10 100	No	No	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Back to Course

5-Implementation of Quick Sort

Started on	Sunday, 12 October 2025, 9:32 AM
State	Finished
Completed on	Sunday, 12 October 2025, 9:41 AM
Time taken	8 mins 21 secs
Marks	1.00/1.00
Grade	10.00 out of 10.00 (100%)

Question 1 | [Correct](#) Mark 1.00 out of 1.00 [Flag question](#)

Write a Program to Implement the Quick Sort Algorithm

Input Format:

The first line contains the no of elements in the list-n

The next n lines contain the elements.

Output:

Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

Answer:

```

1 #include<stdio.h>
2 void swap(int* a, int* b) {
3     int t = *a;
4     *a = *b;
5     *b = t;
6 }
7 void quick(int arr[],int low,int high){
8     if(low<high){
9         int pivot=arr[high];
10        int i=(low-1);
11        for(int j=low;j<=high-1;j++){
12            if(arr[j]<pivot){
13                i++;
14                swap(&arr[i],&arr[j]);
15            }
16        }
17        swap(&arr[i+1],&arr[high]);
18        int pi=i+1;
19        quick(arr,low,pi-1);
20        quick(arr,pi+1,high);
21    }
22 }
23 void printArray(int arr[], int size) {
24     for (int i = 0; i < size; i++) {
25         printf("%d", arr[i]);
26         if (i < size - 1) {
27             printf(" ");
28         }
29     }
30     printf("\n");
31 }
32 int main(){
33     int n;
34     scanf("%d",&n);
35     int arr[n];
36     for(int i=0;i<n;i++)
37         scanf("%d",&arr[i]);
38     quick(arr, 0, n - 1);
39     printArray(arr, n);
40     return 0;
41 }
```

	Input	Expected	Got	
	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	
	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	
	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Finish review](#)

[Back to Course](#)